

Section A: Multiple Choice Questions (MCQs)

1. What does RDD stand for?

- a) Resilient Distributed Dataset
- b) Reliable Data Distribution
- c) Random Data Duplication
- d) Resilient Data Definition

Ans : a)

2. RDDs are:

- a) Mutable and can be changed in place
- b) Immutable - transformations create new RDDs
- c) Only usable with SQL
- d) Automatically persisted in memory by default

Ans : b)

3. Which of the following is a *transformation*, not an *action*?

- a) collect()
- b) count()
- c) map()
- d) reduce()

Ans: c)

4. Spark transformations are:

- a) Eagerly evaluated immediately when called
- b) Lazily evaluated - executed only when an action is triggered
- c) Executed only on the driver node
- d) Not part of the DAG

Ans: b)

5. Which operation is a *wide* transformation (causes a shuffle)?

- a) map()
- b) filter()
- c) groupByKey()
- d) union()

Ans: c)

Hints: A **wide transformation** requires data to be **reshuffled across partitions**, which means Spark has to move data between executors.

6. Which of these creates an RDD from an existing Scala/Python collection?

- a) sc.textFile()
- b) sc.parallelize()

- c) `sc.wholeTextFiles()`
- d) `sc.hadoopFile()`

Ans: a)

7. Which persistence level stores data in memory only, and recomputes from lineage if it doesn't fit? a) MEMORY_AND_DISK b) DISK_ONLY c) MEMORY_ONLY d) OFF_HEAP

Ans: c)

Hints: Stores partitions only in RAM.

- If some partitions **do not fit in memory**, Spark **does not write them to disk**.
- Instead, those partitions are **recomputed from the lineage** when needed.

8. What is the default number of partitions when reading a file with `sc.textFile()` (assuming no second argument given)?

- a) Always exactly 1
- b) Determined by the number of executors only
- c) Based on HDFS block size / input splits (or `defaultMinPartitions`)
- d) Always equal to number of cores on the driver

Ans : c)

11. Which action returns the entire RDD content to the driver program as a local collection?

- a) `take(n)`
- b) `collect()`
- c) `saveAsTextFile()`
- d) `foreach()`

Ans: b)

12. What happens if you call `collect()` on a very large RDD that doesn't fit in driver memory?

- a) Spark automatically writes it to disk
- b) It throws an `OutOfMemoryError` on the driver
- c) It silently truncates the data
- d) Nothing , Spark handles it transparently

Ans : b)

13. `persist()` / `cache()` on an RDD is useful when:

- a) The RDD is used only once in the program
- b) The RDD is reused multiple times across actions, to avoid recomputation
- c) You want to make the RDD mutable
- d) You want to force a shuffle

Ans: b)

14. What does `flatMap()` do differently from `map()`?

- a) `flatMap()` cannot take a lambda/function
- b) `flatMap()` flattens the output , each input can map to zero, one, or many output elements
- c) `flatMap()` only works on numeric RDDs
- d) There is no difference

Ans: b)

15. Which of these is NOT a valid way to create an RDD?

- a) From a collection using parallelize()
- b) From an external file using textFile()
- c) By transforming an existing RDD
- d) By directly instantiating new RDD() with data in PySpark

Ans: d)

Section B: Worked Examples (map & filter only)

Example 1 — map(): double every number

Output: [2, 4, 6, 8, 10]

Example 2 — filter(): keep only even numbers

Output: [2, 4, 6]

Example 3 — chaining map() and filter()

numbers = sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8])

#todo

Output: [18, 21, 24] (6*3=18, 7*3=21, 8*3=24)

Ans: In Pyspark:

```
rdd1 = sc.parallelize ([6,7,8])  
rdd2 = rdd1.map(lambda x: x*3)  
rdd2.collect ()
```

Ans: In Spark-Shell

```
var rdd1 = sc.parallelize ([6,7,8])  
var rdd2 = rdd1.map(x=>x*3)  
rdd2. collect ()
```

Example 4 — map() on strings

Output: ['ALICE', 'BOB', 'CHARLIE']

Ans : In Pyspark:

```
var rdd 1 = sc.textFile ("Location")  
var rdd1_Upper = rdd1.map(lambda x: x.Upper( ))  
rdd1_Upper.collect ()
```

Ans: In Spark-Shell

```
rdd1 = sc.textFile ("Location")  
rdd2_Upper = rdd1.map(x => x.toUpperCase())  
rdd2_Upper.collect ()
```

Example 5 — filter() on strings

Find all the words with length greater than 5 from a list of words.

Output: ['apple', 'banana', 'watermelon']

Ans : In Pyspark:

```
rdd 1 = sc.textFile ("Location")  
rdd1_WordsWL5 = rdd1.filter(lambda x: x len(x) >= 5)  
rdd1_WordsWL5.collect ()
```

Ans: In Spark-Shell

```
var rdd1 = sc.textFile ("Location")  
var rdd2_wordsWL5 = rdd1.filter(x => x.length >= 5)  
rdd2._wordsWL5 .collect ()
```

Section C: Scenario-Based Coding Questions

For each scenario, write the PySpark (or Scala) code. Assume sc is the SparkContext and, where relevant, name your variables clearly. **Use only map(), filter(), flatMap(), and basic actions (collect(), count(), sum(), first(), take(), max(), min()) — no reduceByKey, groupByKey, or aggregateByKey needed for these.**

Scenario 1 — Splitting Lines into Words You have a text file sample.txt with multiple lines of text. Write RDD code to: a) Read the file into an RDD of lines b) Split each line into individual words and

produce a single flat RDD of all words (hint: flatMap) c) Convert every word to lowercase d) Count how many words are longer than 4 characters

Ans : In Pyspark:

```
# a) Read the file into an RDD of lines
lines_rdd = sc.textFile("sample.txt")

# b) Split each line into words (using flatMap)
words_rdd = lines_rdd.flatMap(lambda line: line.split())

# c) Convert every word to lowercase
lower_words_rdd = words_rdd.map(lambda word: word.lower())

# d) Count words longer than 4 characters
long_words = lower_words_rdd.filter(lambda word: len(word) > 4)
count_long_words = long_words.count()

words_rdd.collect()
lower_words_rdd.collect()
long_words.collect()
print(count_long_words)
```

Ans: In Spark-Shell

```
# a) Read the file into an RDD of lines
var lines_rdd = sc.textFile("sample.txt")

# b) Split lines into individual words
var words_rdd = lines_rdd.flatMap(line => line.split(" "))

# c) Convert every word to lowercase
var lowercase_rdd = words_rdd.map(word => word.toLowerCase())

# d) Count words longer than 4 characters
var countLongWords = lowercase_rdd.filter(word => word.length > 4).count()

words_rdd.collect()
lowercase_rdd.collect()
print(countLongWords)
```

Scenario 2 — Filtering and Counting You have an RDD of integers: numbers = sc.parallelize([4, 7, 2, 9, 15, 22, 3, 18, 11]). Write code to: a) Return only the even numbers b) Count how many numbers are greater than 10 c) Find the sum of all numbers

Ans : In Pyspark:

```
# a) Return only even numbers
Int_rdd = sc.parallelize([4,7,2,9,15,22,3,18,11])
even_rdd = Int_rdd.filter(lambda x: x %2 == 0)

# b) Count numbers which are greater than 10
Greater10_rdd = Int_rdd.filter (lambda x: x >10)
count_greater10= Greater10_rdd.count()

# c) Find the sum of all numbers
sum_rdd = Int_rdd.sum()

# Display Results
even_rdd.collect()
Greater10_rdd.collect()
print(count_greater10)
```

Ans: In Spark-Shell

```
# a) Return only even numbers
var Int_rdd = sc.parallelize(Array(4,7,2,9,15,22,3,18,11))
var even_rdd = Int_rdd.filter(x=> x % 2 == 0 )

# b) Count numbers which are greater than 10
var Greater10_rdd = Int_rdd.filter (x=> x>10)
count_greater10= Greater10_rdd.count()

# c) Find the sum of all numbers
var sum_rdd = Int_rdd.sum()

even_rdd.collect()
Greater10_rdd.collect()
print(count_greater10)
```

Scenario 3 — Sales Data Filtering You have an RDD of tuples representing sales: (region, amount), e.g. sales = sc.parallelize([("East",100), ("West",200), ("East",150), ("North",50), ("West",300)]) Write

code to: a) Extract just the amount values into a new RDD using map() b) Filter to keep only sales where amount > 100 c) Find the total of all filtered amounts and the single highest amount overall (Note: computing totals *per region* would normally use reduceByKey — we'll cover that later. For now, just work with the amounts as a whole.)

Ans : In Pyspark:

Create the RDD

```
sales = sc.parallelize([
```

```
    ("East", 100),
```

```
    ("West", 200),
```

```
    ("East", 150),
```

```
    ("North", 50),
```

```
    ("West", 300)
```

```
])
```

a) Extract just the amount values

```
amounts_rdd = sales.map(lambda x: x[1])
```

b) Keep only sales where amount > 100

```
filtered_sales_rdd = sales.filter(lambda x: x[1] > 100)
```

Extract filtered amounts

```
filtered_amounts_rdd = filtered_sales_rdd.map(lambda x: x[1])
```

c) Total of filtered amounts

```
total_filtered = filtered_amounts_rdd.sum()
```

Single highest amount overall

```
highest_amount = filtered_amounts_rdd.max()
```

Display results

```
print(amounts_rdd.collect())
```

```
print(filtered_sales_rdd.collect())
```

```
print(total_filtered)
```

```
print(highest_amount)
```

Ans: In Spark-Shell

```
# Create the RDD

var sales = sc.parallelize(Array(

  ("East", 100),

  ("West", 200),

  ("East", 150),

  ("North", 50),

  ("West", 300)

))

# a) Extract just the amount values

var amounts_rdd = sales.map(x => x._2)

# b) Keep only sales where amount > 100

var filteredSales_rdd = sales.filter(x => x._2 > 100)

# Extract filtered amounts

var filteredAmounts_rdd = filteredSales_rdd.map(x => x._2)

# c) Total of filtered amounts

var totalFiltered = filteredAmounts_rdd.sum()

# Single highest amount overall

var highestAmount = amounts_rdd.max()
```

Scenario 4 — Joining Two Datasets You have two RDDs:

```
employees = sc.parallelize([(101,"Alice"), (102,"Bob"), (103,"Charlie")])

salaries = sc.parallelize([(101,50000), (102,60000), (104,45000)])
```

Write code to: a) Perform an inner join on employee ID and display (id, name, salary) b) Perform a left outer join from employees and explain what happens to employee 103 and 104 in each case.

Ans: In Pyspark:

```
employees = sc.parallelize([

  (101, "Alice"),

  (102, "Bob"),

  (103, "Charlie")
```



```

])
salaries = sc.parallelize([
    (101, 50000),
    (102, 60000),
    (104, 45000)
])
# Perform inner_join
inner_join = employees.join(salaries)
result = inner_join.map(lambda x: (x[0], x[1][0], x[1][1]))
result.collect()

```

#b) Perform a left outer join from employees and explain what happens to employee 103 and 104 in each case.

```

left_join = employees.leftOuterJoin(salaries)
result_left = left_join.map( lambda x: (x[0], x[1][0], x[1][1]))
result_left.collect()

```

Ans: Spark-shell

```

var employees = sc.parallelize(Array(
    (101,"Alice"),
    (102,"Bob"),
    (103,"Charlie")
))
var salaries = sc.parallelize(Array(
    (101,50000),
    (102,60000),
    (104,45000)
))
# Inner Join
var innerJoin = employees.join(salaries)
innerJoin.collect()
# Left Outerjoin

```

```
var leftJoin = employees.leftOuterJoin(salaries)

leftJoin.collect()
```

Scenario 5 — Lazy Evaluation Trace Given the following code, list the sequence of actions/transformations and state **at which exact line Spark actually starts computing anything**, and why.

```
rdd1 = sc.parallelize([1,2,3,4,5])    : Create rdd
rdd2 = rdd1.map(lambda x: x * 2)      transformation (map)
rdd3 = rdd2.filter(lambda x: x > 4)   transformation (filter)
print("Defined transformations")      python operation
result = rdd3.collect()               action (collect) - Here actually spark starts computing
print(result)                         python output
```

Scenario 6 — map() vs filter(): Picking the Right Transformation You have temps = sc.parallelize([21, 35, 40, 15, 28, 33, 8]) representing daily temperatures (°C). a) Write code using map() to convert every value to Fahrenheit ($F = C * 9/5 + 32$) b) Write code using filter() to keep only days where the Celsius temperature was above 30 c) In your own words, explain the difference between what map() returns vs what filter() returns (size of output RDD, type of output).

Ans: In pyspark

```
# create rdd

temps = sc.parallelize([21, 35, 40, 15, 28, 33, 8])

fahrenheit_rdd = temps.map(lambda c: c * 9/5 + 32)

fahrenheit_rdd.collect()

above30_rdd = temps.filter(lambda c: c > 30)

above30_rdd.collect()

map( )
```

- Transforms every element in the RDD.
- The output RDD usually has the same number of elements as the input RDD.
- The type of output can change.

Filter()

- Filter out the elements which satisfy the conditions
- The output RDD usually has the less number of elements as the input RDD.
- The type of output usually remains same.

Scenario 7— Pass/Fail Filtering Given scores = sc.parallelize([("Alice",90), ("Bob",42), ("Charlie",70), ("David",35), ("Eve",100)]) (name, score out of 100, pass mark = 40): a) Write code using filter() to return only the students who passed b) Write code using map() to convert the list into (name, "Pass") or (name, "Fail") pairs c) Count how many students failed (*Grouping/averaging by subject would use aggregateByKey — that's for a later session.*)

Ans: Pyspark

```
scores = sc.parallelize([
    ("Alice", 90),
    ("Bob", 42),
    ("Charlie", 70),
    ("David", 35),
    ("Eve", 100)
])

passed_students = scores.filter(lambda x: x[1] >= 40)

passed_students.collect()

pass_fail_rdd = scores.map(lambda x: (x[0], "Pass" if x[1] >= 40 else "Fail"))

pass_fail_rdd.collect()

failed_count = scores.filter(lambda x: x[1] < 40).count()

print(failed_count)
```
